arXiv:1906.01624v2 [cs.LG] 20 Jun 2019

# Off-Policy Evaluation via Off-Policy Classification

**Alex Irpan**[1], **Kanishka Rao**[1], **Konstantinos Bousmalis**[2],
**Chris Harris**[1], **Julian Ibarz**[1], **Sergey Levine**[1,3]

[1]Google Brain, Mountain View, USA
[2]DeepMind, London, UK
[3]University of California Berkeley, Berkeley, USA

{alexirpan,kanishkarao,konstantinos,ckharris,julianibarz,slevine}@google.com

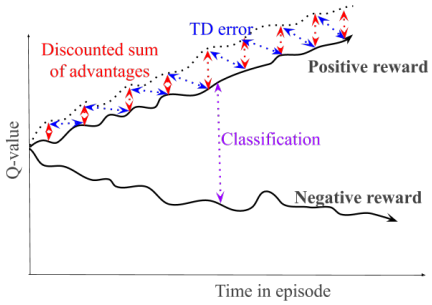
## Abstract

In this work, we consider the problem of model selection for deep reinforcement learning (RL) in real-world environments. Typically, the performance of deep RL algorithms is evaluated via on-policy interactions with the target environment. However, comparing models in a real-world environment for the purposes of early stopping or hyperparameter tuning is costly and often practically infeasible. This leads us to examine off-policy policy evaluation (OPE) in such settings. We focus on OPE for value-based methods, which are of particular interest in deep RL, with applications like robotics, where off-policy algorithms based on Q-function estimation can often attain better sample complexity than direct policy optimization. Existing OPE metrics either rely on a model of the environment, or the use of importance sampling (IS) to correct for the data being off-policy. However, for high-dimensional observations, such as images, models of the environment can be difficult to fit and value-based methods can make IS hard to use or even ill-conditioned, especially when dealing with continuous action spaces. In this paper, we focus on the specific case of MDPs with continuous action spaces and sparse binary rewards, which is representative of many important real-world applications. We propose an alternative metric that relies on neither models nor IS, by framing OPE as a positive-unlabeled (PU) classification problem with the Q-function as the decision function. We experimentally show that this metric outperforms baselines on a number of tasks. Most importantly, it can reliably predict the relative performance of different policies in a number of generalization scenarios, including the transfer to the real-world of policies trained in simulation for an image-based robotic manipulation task.

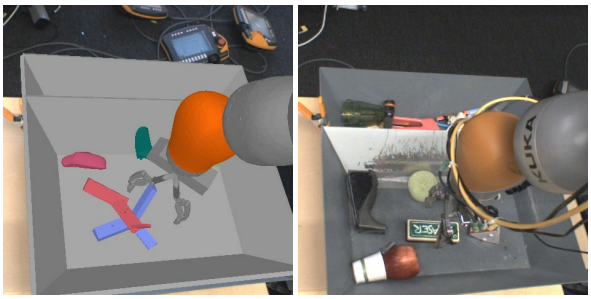
## 1 Introduction

Supervised learning has seen significant advances in recent years, in part due to the use of large, standardized datasets [5]. When researchers can evaluate real performance of their methods on the same data via a standardized offline metric, the progress of the field can be rapid. Unfortunately, such metrics have been lacking in reinforcement learning (RL). Model selection and performance evaluation in RL are typically done by estimating the average on-policy return of a method in the target environment. Although this is possible in most simulated environments [2, 3, 35], real-world environments, like in robotics, make this difficult and expensive [34]. Off-policy evaluation (OPE) has the potential to change that: a robust off-policy metric could be used together with realistic and complex data to evaluate the expected performance of off-policy RL methods, which would enable

Preprint. Under review.




(a) Visual summary of off-policy metrics



(b) Robotics grasping simulation-to-reality gap.

Figure 1: **(a) Visual illustration of our method:** We propose using classification-based approaches to do off-policy evaluation. Solid curves represent $Q(\mathbf{s}, \mathbf{a})$ over a positive and negative trajectory, with the dashed curve representing $\max_{\mathbf{a}'} Q(\mathbf{s}, \mathbf{a}')$ along states the positive trajectory visits (the corresponding negative curve is omitted for simplicity). Baseline approaches (blue, red) measure Q-function fit between $Q(\mathbf{s}, \mathbf{a})$ to $\max_{\mathbf{a}'} Q(\mathbf{s}, \mathbf{a}')$. Our approach (purple) directly measures separation of $Q(\mathbf{s}, \mathbf{a})$ between positive and negative trajectories. **(b) The visual "reality gap" of our most challenging task:** off-policy evaluation of the generalization of image-based robotic agents trained solely in simulation (left) using historical data from the target real-world environment (right).

rapid progress on important real-world RL problems. Furthermore, it would greatly simplify transfer learning in RL, where OPE would enable model selection and algorithm design in simple domains (e.g., simulation) while evaluating the performance of these models and algorithms on complex domains (e.g., using previously collected real-world data).

Previous approaches to off-policy evaluation [6, 12, 26, 33] generally use importance sampling (IS) or learned dynamics models. However, this makes them difficult to use with many modern deep RL algorithms. First, OPE is most useful in the off-policy RL setting, where we expect to use real-world data as the "validation set", but many of the most commonly used off-policy RL methods are based on value function estimation, produce deterministic policies [19, 36], and do not require any knowledge of the policy that generated the real-world training data. This makes them difficult to use with IS. Furthermore, many of these methods might be used with high-dimensional observations, such as images. Although there has been considerable progress in predicting future images [1, 18], learning sufficiently accurate models in image space for effective evaluation is still an open research problem. We therefore aim to develop an OPE method that requires neither IS nor models.

We observe that for model selection, it is sufficient to predict some statistic *correlated* with policy return, rather than directly predict policy return. We address the specific case of binary-reward MDPs: tasks where the agent receives a non-zero reward only once during an episode, at the final timestep (Sect. 2). These can be interpreted as tasks where the agent can either "succeed" or "fail" in each trial, and although they form a subset of all possible MDPs, this subset is quite representative of many real-world tasks, and is actively used e.g. in robotic manipulation [14, 29]. The novel contribution of our method (Sect. 3) is to frame OPE as a positive-unlabeled (PU) classification [15] problem, which provides for a way to derive OPE metrics that are both *(a)* fundamentally different from prior methods based on IS and model learning, and *(b)* perform well in practice on both simulated and real-world tasks. Additionally, we identify and present (Sect. 4) a list of generalization scenarios in RL that we would want our metrics to be robust against. We experimentally show (Sect. 6) that our suggested OPE metrics outperform a variety of baseline methods across all of the evaluation scenarios, including a simulation-to-reality transfer scenario for a vision-based robotic grasping task (see Fig. 1b).

## 2 Preliminaries

We focus on finite–horizon, deterministic Markov decision processes (MDP). We define an MDP as $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{S}_0, r, \gamma)$. $\mathcal{S}$ is the state–space, $\mathcal{A}$ the action–space, and both can be continuous. $\mathcal{P}$ defines transitions to next states given the current state and action, $\mathcal{S}_0$ defines initial state distribution, $r$ is the reward function, and $\gamma \in [0, 1]$ is the discount factor. Episodes are of finite length $T$: at a given

time-step $t$ the agent is at state $\mathbf{s}_t \in \mathcal{S}$, samples an action $\mathbf{a}_t \in \mathcal{A}$ from a policy $\pi$, receives a reward $r_t = r(\mathbf{s}_t, \mathbf{a}_t)$, and observes the next state $\mathbf{s}_{t+1}$ as determined by $\mathcal{P}$.

The goal in RL is to learn a policy $\pi(\mathbf{a}_t|\mathbf{s}_t)$ that maximizes the expected episode return $R(\pi) = \mathbb{E}_\pi[\sum_{t=0}^{T} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t)]$. A value of a policy for a given state $\mathbf{s}_t$ is defined as $V^\pi(\mathbf{s}_t) = \mathbb{E}_\pi[\sum_{t'=t}^{T} \gamma^{t'-t} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}^\pi)]$ where $\mathbf{a}_t^\pi$ is the action $\pi$ takes at state $\mathbf{s}_t$ and $\mathbb{E}_\pi$ implies an expectation over trajectories $\tau = (\mathbf{s}_1, \mathbf{a}_1, \ldots, \mathbf{s}_T, \mathbf{a}_T)$ sampled from $\pi$. Given a policy $\pi$, the expected value of its action $a_t$ at a state $s_t$ is called the Q-value and is defined as $Q^\pi(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}_\pi[r(\mathbf{s}_t, \mathbf{a}_t) + V^\pi(\mathbf{s}_{t+1})]$.

We assume the MDP is a binary reward MDP, which satisfies the following properties: transitions $\mathcal{P}$ are deterministic, $\gamma = 1$, the reward is $r_t = 0$ at all intermediate steps, and the final reward $r_T$ is in $\{0, 1\}$, indicating whether the final state is a failure or a success. We learn Q-functions $Q(\mathbf{s}, \mathbf{a})$ and aim to evaluate policies $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$.

### 2.1 Positive-unlabeled learning

Positive-unlabeled (PU) learning is a set of techniques for learning binary classification from partially labeled data, where we have many unlabeled points and some positively labeled points [15]. We will make use of these ideas in developing our OPE metric. Positive-unlabeled data is sufficient to learn a binary classifier if the positive class prior $p(y = 1)$ is known.

Let $(X, Y)$ be a labeled binary classification problem, where $Y = \{0, 1\}$. Let $g : X \to \mathbb{R}$ be some decision function, and let $\ell : \mathbb{R} \times \{0, 1\} \to \mathbb{R}$ be our loss function. Suppose we want to evaluate loss $\ell(g(x), y)$ over negative examples $(x, y = 0)$, but we only have unlabeled points $x$ and positively labeled points $(x, y = 1)$. The key insight of PU learning is that the loss over negatives can be indirectly estimated from $p(y = 1)$. For any $x \in X$,

$$p(x) = p(x|y = 1)p(y = 1) + p(x|y = 0)p(y = 0) \tag{1}$$

It follows that for any $f(x)$, $\mathbb{E}_{X,Y}[f(x)] = p(y = 1)\mathbb{E}_{X|Y=1}[f(x)] + p(y = 0)\mathbb{E}_{X|Y=0}[f(x)]$, since by definition $\mathbb{E}_X[f(x)] = \int_x p(x)f(x)dx$. Letting $f(x) = \ell(g(x), 0)$ and rearranging gives

$$p(y = 0)\mathbb{E}_{X|Y=0}[\ell(g(x), 0)] = \mathbb{E}_{X,Y}[\ell(g(x), 0)] - p(y = 1)\mathbb{E}_{X|Y=1}[\ell(g(x), 0)] \tag{2}$$

In Sect. 3, we reduce off-policy evaluation of a policy $\pi$ to a classification problem, provide reasoning for how to estimate $p(y = 1)$, use PU learning to estimate classification error with Eqn. 2, then use the error to estimate a lower bound on return $R(\pi)$ with Theorem 1.

## 3 Off-policy evaluation via state-action pair classification

A Q-function $Q(\mathbf{s}, \mathbf{a})$ predicts the expected return of each action $\mathbf{a}$ given state $\mathbf{s}$. The policy $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$ can be viewed as a classifier that predicts the best action. We propose an off-policy evaluation method connecting off-policy evaluation to estimating validation error for a positive-unlabeled (PU) classification problem [15]. Our metric can be used with Q-function estimation methods without requiring importance sampling, and can be readily applied in a scalable way to image-based deep RL tasks.

We present an analysis for binary reward MDPs, defined in Sec. 2. In a binary reward MDP, the deterministic dynamics means each $(\mathbf{s}_t, \mathbf{a}_t)$ is either potentially effective, or guaranteed to lead to failure.

**Definition 1.** In a binary reward MDP, $(\mathbf{s}_t, \mathbf{a}_t)$ is **effective** if an optimal policy $\pi^*$ can achieve success, i.e an episode return of 1, after taking $\mathbf{a}_t$ in $\mathbf{s}_t$. Equivalently, there exists a sequence of future actions that reaches a success state. $(\mathbf{s}_t, \mathbf{a}_t)$ is **catastrophic** if no such sequence exists.

Under this definition, the return of a trajectory $\tau$ is 1 if and only if all $(\mathbf{s}_t, \mathbf{a}_t)$ in $\tau$ are effective (see Appendix A.1), the label for $(\mathbf{s}_t, \mathbf{a}_t)$ does not depend on the policy we are evaluating, and the classification error of $\pi$ at each time $t$ can be used to bound return $R(\pi)$.

**Theorem 1.** *Given a binary reward MDP and a policy $\pi$, let $\rho^+_{t,\pi}$ denote the state distribution at time t, given that $\pi$ was followed and all its previous actions $\mathbf{a}_1, \cdots, \mathbf{a}_{t-1}$ were effective. Let $\mathcal{A}_-(\mathbf{s})$ denote the set of catastrophic actions at state $\mathbf{s}$, and let $\epsilon_t = \mathbb{E}_{\rho^+_{t,\pi}}\left[\sum_{\mathbf{a}\in\mathcal{A}_-(\mathbf{s}_t)} \pi(\mathbf{a}|\mathbf{s}_t)\right]$ be the*

*per-step expectation of $\pi$ making its first mistake at time $t$, with $\epsilon = \frac{1}{T}\sum_{i=1}^{T}\epsilon_t$ being average error over all $(\mathbf{s}_t, \mathbf{a}_t)$. Then $R(\pi) \geq 1 - T\epsilon$, and this lower bound is tight.*

*Proof.* Failure rate $1 - R(\pi)$ is the total probability $\pi$ makes its first mistake at time $t$, summed over all $t$. For each $t$, the probability that $\pi$ follows effective actions $\mathbf{a}_1, \cdots, \mathbf{a}_{t-1}$ then a catastrophic action $\mathbf{a}_t$ is $\epsilon_t \prod_{i=1}^{t-1}(1-\epsilon_i)$, giving $1 - R(\pi) = \sum_{t=1}^{T}\epsilon_t \prod_{i=1}^{t-1}(1-\epsilon_i) \leq \sum_{t=1}^{T}\epsilon_t \leq T\epsilon$. This gives $R(\pi) \geq 1 - T\epsilon$, which is tight when $\epsilon_1 = \cdots = \epsilon_{T-1} = 0, \epsilon_T = T\epsilon$. □

An alternative proof in Appendix A.3 is based on imitation learning behavioral cloning bounds from Ross & Bagnell [30].

A smaller $\epsilon$ gives a higher lower bound on return, which implies a better $\pi$. This leaves estimating $\epsilon$. The primary challenge with this approach is that we do not have negative labels – that is, for trials that receive a return of 0 in the validation set, we do not know which $(\mathbf{s}, \mathbf{a})$ were in fact catastrophic, and which were recoverable. We discuss how we address this problem next.

### 3.1 Missing negative labels

Recall that $(\mathbf{s}_t, \mathbf{a}_t)$ is effective if $\pi^*$ can succeed from $\mathbf{s}_{t+1}$. Since $\pi^*$ is at least as good as $\pi_b$, whenever $\pi_b$ succeeds, all tuples $(\mathbf{s}_t, \mathbf{a}_t)$ in the trajectory $\tau$ must be effective. However, the converse is not true, since $\pi^*$ could succeed from $(\mathbf{s}_t, \mathbf{a}_t)$ where $\pi_b$ failed. This is an instance of the positive-unlabeled (PU) learning problem from Sect. 2.1, where $\pi_b$ positively labels some $(\mathbf{s}, \mathbf{a})$ and the remaining are unlabeled. In the RL setting, $X = \mathcal{S} \times \mathcal{A}$, labels $\{0, 1\} = \{\text{catastrophic, effective}\}$, and a natural choice for decision function $g$ is $g(\mathbf{s}, \mathbf{a}) = Q(\mathbf{s}, \mathbf{a})$, since $Q(\mathbf{s}, \mathbf{a})$ should be high for effective $(\mathbf{s}, \mathbf{a})$ and low for catastrophic $(\mathbf{s}, \mathbf{a})$.

We aim to estimate $\epsilon$, the probability that $\pi$ takes a catastrophic action – i.e., that $(\mathbf{s}, \pi(\mathbf{s}))$ is a false positive. Note that if $(\mathbf{s}, \pi(\mathbf{s}))$ is predicted to be catastrophic, but is actually effective, this false-negative does not impact future reward – since the action is effective, there is still a path to reach success. We want just the false-positive risk, $\epsilon = p(y=0)\mathbb{E}_{X|Y=0}\left[\ell(g(x), 0)\right]$. This is the same as Eqn. 2, and using $g(\mathbf{s}, \mathbf{a}) = Q(\mathbf{s}, \mathbf{a})$ gives

$$\epsilon = \mathbb{E}_{(\mathbf{s},\mathbf{a})}\left[\ell(Q(\mathbf{s}, \mathbf{a}), 0)\right] - p(y=1)\mathbb{E}_{(\mathbf{s},\mathbf{a}),y=1}\left[\ell(Q(\mathbf{s}, \mathbf{a}), 0)\right]. \tag{3}$$

Eqn. 3 is the core of all our proposed metrics. While it might at first seem that the class prior $p(y=1)$ should be task-dependent, recall that the error $\epsilon_t$ is the expectation over the state distribution $\rho^+_{t,\pi}$, where the actions $\mathbf{a}_1, \cdots, \mathbf{a}_{t-1}$ were all effective. This is equivalent to following an optimal "expert" policy $\pi^*$, and although we are estimating $\epsilon_t$ from data generated by behavior policy $\pi_b$, we should match the positive class prior $p(y=1)$ we would observe from expert $\pi^*$. Assuming the task is feasible, meaning that the policy has effective actions available from the start, we have $R(\pi^*) = 1$. Therefore, although the validation dataset will likely have both successes and failures, a prior of $p(y=1) = 1$ is the ideal prior, and this holds independently of the environment. We illustrate this further with a didactic example in Sect. 6.1.

Theorem 1 relies on estimating $\epsilon$ over the distribution $\rho^+_{t,\pi}$, but our dataset $\mathcal{D}$ is generated by an unknown behavior policy $\pi_b$. A natural approach here would be importance sampling (IS) [6], but: *(a)* we assume no knowledge of $\pi_b$, and *(b)* IS is not well-defined for deterministic policies $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. Another approach is to subsample $\mathcal{D}$ to transitions $(\mathbf{s}, \mathbf{a})$ where $\mathbf{a} = \pi(\mathbf{s})$ [20]. This ensures an on-policy evaluation, but can encounter finite sample issues if $\pi_b$ does not sample $\pi(\mathbf{s})$ frequently enough. Therefore, we assume classification error over $\mathcal{D}$ is a good enough proxy that correlates well with classification error over $\rho^+_{t,\pi}$. This is admittedly a strong assumption, but empirical results in Sect. 6 show surprising robustness to distributional mismatch. This assumption is reasonable if $\mathcal{D}$ is broad (e.g., generated by a sufficiently random policy), but may produce pessimistic estimates when potential effective actions in $\mathcal{D}$ are unlabeled.

### 3.2 Off-policy classification for OPE

Based off of the derivation from Sect. 3.1, our proposed off-policy classification (OPC) score is defined by the negative loss when $\ell$ in Eqn. 3 is the 0-1 loss. Let $b$ be a threshold, with $\ell(Q(\mathbf{s}, \mathbf{a}), Y) = \frac{1}{2} + \left(\frac{1}{2} - Y\right)\text{sign}(Q(\mathbf{s}, \mathbf{a}) - b)$. This gives

$$\text{OPC}(Q) = p(y=1)\mathbb{E}_{(\mathbf{s},\mathbf{a}),y=1}\left[1_{Q(\mathbf{s},\mathbf{a})>b}\right] - \mathbb{E}_{(\mathbf{s},\mathbf{a})}\left[1_{Q(\mathbf{s},\mathbf{a})>b}\right]. \tag{4}$$

To be fair to each $Q(\mathbf{s}, \mathbf{a})$, threshold $b$ is set separately for each Q-function to maximize OPC($Q$). Given $N$ transitions and $Q(\mathbf{s}, \mathbf{a})$ for all $(\mathbf{s}, \mathbf{a}) \in \mathcal{D}$, the best $b$ for each $Q(\mathbf{s}, \mathbf{a})$ can be computed by sorting all Q-values, then scanning every threshold $b$, which takes $O(N \log N)$ time per Q-function (see Appendix B). This avoids favoring Q-functions that systematically overestimate or underestimate the true value.

Alternatively, $\ell$ can be a soft loss function. We experimented with $\ell(Q(\mathbf{s}, \mathbf{a}), Y) = (1 - 2Y)Q(\mathbf{s}, \mathbf{a})$, which is minimized when $Q(\mathbf{s}, \mathbf{a})$ is large for $Y = 1$ and small for $Y = 0$. The negative of this loss is called the SoftOPC.

$$\text{SoftOPC}(Q) = p(y = 1)\mathbb{E}_{(\mathbf{s},\mathbf{a}),y=1}\left[Q(\mathbf{s}, \mathbf{a})\right] - \mathbb{E}_{(\mathbf{s},\mathbf{a})}\left[Q(\mathbf{s}, \mathbf{a})\right]. \tag{5}$$

If episodes have different lengths, to avoid focusing on long episodes, transitions $(\mathbf{s}, \mathbf{a})$ from an episode of length $T$ are weighted by $\frac{1}{T}$ when estimating OPC and its variants.

### 3.3 Evaluating OPE metrics

The standard evaluation method for OPE is to report MSE to the true episode return [20, 33]. However, our metrics do not estimate episode return directly. The OPC($Q$)'s estimate of $\epsilon$ will differ from the true value, since it is estimated over our dataset $\mathcal{D}$ instead of over the distribution $\rho^{+}_{t,\pi}$. Meanwhile, SoftOPC($Q$) does not estimate $\epsilon$ directly due to using a soft loss function. Despite this, the OPC and SoftOPC are still useful OPE metrics if they *correlate* well with $\epsilon$ or episode return $R(\pi)$.

We propose an alternative evaluation method. Instead of reporting MSE, we train a large suite of Q-functions $Q(\mathbf{s}, \mathbf{a})$ with different learning algorithms, evaluating true return of the equivalent argmax policy for each $Q(\mathbf{s}, \mathbf{a})$, then compare correlation of the metric to true return. We report two correlations, the coefficient of determination $R^2$ of line of best fit, and the Spearman rank correlation $\xi$ [31].[1] $R^2$ measures confidence in how well our linear best fit will predict returns of new models, whereas $\xi$ measures confidence the metric ranks different policies correctly, without assuming a linear best fit.

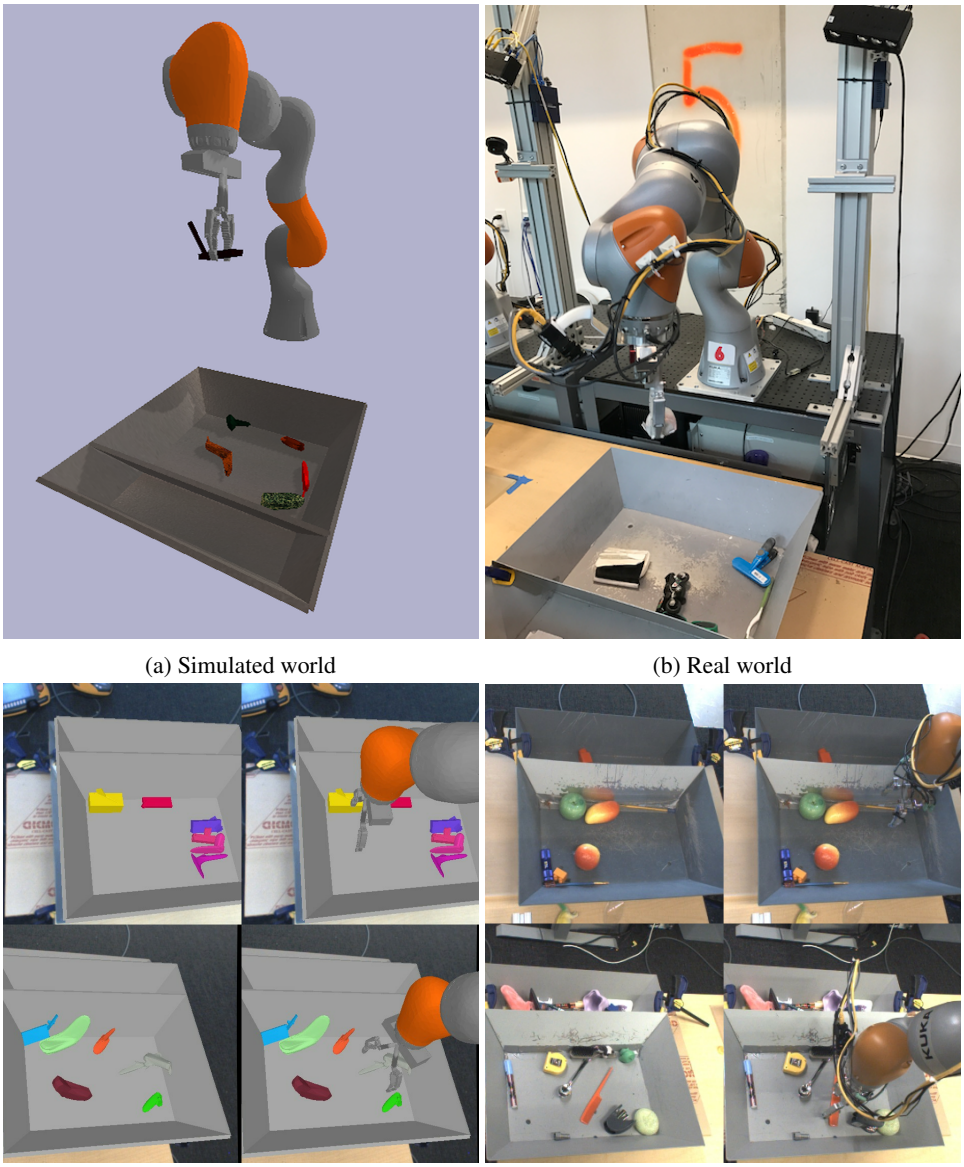
## 4 Applications of OPE for transfer and generalization

Off-policy evaluation (OPE) has many applications. One is to use OPE as an early stopping or model selection criteria when training from off-policy data. Another is applying OPE to validation data collected in another domain to measure generalization to new settings. Several papers [4, 25, 28, 37, 38] have examined overfitting and memorization in deep RL, proposing explicit train-test environment splits as benchmarks for RL generalization. Often, these test environments are defined in simulation, where it is easy to evaluate the policy in the test environment. This is no longer sufficient for real-world settings, where test environment evaluation can be expensive. In real-world problems, off-policy evaluation is an inescapable part of measuring generalization performance in an efficient, tractable way. To demonstrate this, we identify a few common generalization failure scenarios faced in reinforcement learning, applying OPE to each one. When there is *insufficient off-policy training data* and new data is not collected online, models may memorize state-action pairs in the training data. RL algorithms collect new on-policy data with high frequency. If training data is generated in a systematically biased way, we have *mismatched off-policy training data.* The model fails to generalize because systemic biases cause the model to miss parts of the target distribution. Finally, models trained in simulation usually do not generalize to the real-world, due to the *training and test domain gap*: the differences in the input space (see Fig. 1b and Fig. 2) and the dynamics. All of these scenarios are, in principle, identifiable by off-policy evaluation, as long as validation is done against data sampled from the final testing environment. We evaluate our proposed and baseline metrics across all these scenarios.

## 5 Related work

Off-policy policy evaluation (OPE) predicts the return of a learned policy $\pi$ from a fixed off-policy dataset $\mathcal{D}$, generated by one or more behavior policies $\pi_b$. Prior works [6, 9, 12, 20, 26, 32] do so

[1] We slightly abuse notation here, and should clarify that $R^2$ is used to symbolize the coefficient of determination and should not be confused with $R(\pi)$, the average return of a policy $\pi$.



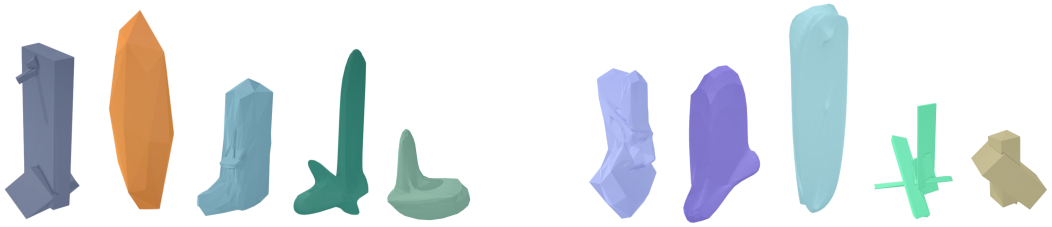
(a) Simulated world (b) Real world

(c) Simulated samples (d) Real samples

Figure 2: **An example of a training and test domain gap.** We display this with a robotic grasping task. *Top row:* The setup we used in (a) simulation and (b) the real-world. *Bottom row:* Images used during training, from (c) simulated grasping over procedurally generated objects; and from (d) the real-world, with a varied collection of everyday physical objects.

with importance sampling (IS) [10], MDP modeling, or both. Importance sampling requires querying $\pi(\mathbf{a}|\mathbf{s})$ and $\pi_b(\mathbf{a}|\mathbf{s})$ for any $\mathbf{s} \in \mathcal{D}$, to correct for the shift in state-action distributions. In RL, the cumulative product of IS weights along $\tau$ is used to weight its contribution to $\pi$'s estimated value [26]. Several variants have been proposed, such as step-wise IS and weighted IS [21]. In MDP modeling, a model is fitted to $\mathcal{D}$, and $\pi$ is rolled out in the learned model to estimate average return [12, 22]. The performance of these approaches is worse if dynamics or reward are poorly estimated, which tends to occur for image-based tasks. Improving these models is an active research question [1, 18]. State of the art methods combine IS-based estimators and model-based estimators using doubly robust estimation and ensembles to produce improved estimators with theoretical guarantees [6, 7, 9, 12, 33].

These IS and model-based OPE approaches assume importance sampling or model learning are feasible. This assumption often breaks down in modern deep RL approaches. When $\pi_b$ is unknown, $\pi_b(\mathbf{a}|\mathbf{s})$ cannot be queried. When doing value-based RL with deterministic policies, $\pi(\mathbf{a}|\mathbf{s})$ is



(a) Train objects (b) Test objects

Figure 3: **Example of mismatched off-policy training data.** Train objects and test objects from simulated grasping task in Sect. 6.2. Given large amounts of data from train objects, models do not fully generalize to test objects.

undefined for off-policy actions. When working with high-dimensional observations, model learning is often too difficult to learn a reliable model for evaluation.

Many recent papers [4, 25, 28, 37, 38] have defined train-test environment splits to evaluate RL generalization, but define test environments in simulation where there is no need for OPE. We demonstrate how OPE provides tools to evaluate RL generalization for real-world environments. While to our knowledge no prior work has proposed a classification-based OPE approach, several prior works have used supervised classifiers to predict transfer performance from a few runs in the test environment [16, 17]. To our knowledge, no other OPE papers have shown results for large image-based tasks where neither importance sampling nor model learning are viable options.

**Baseline metrics** Since we assume importance-sampling and model learning are infeasible, many common OPE baselines do not fit our problem setting. In their place, we use other Q-learning based metrics that also do not need importance sampling or model learning and only require a $Q(\mathbf{s}, \mathbf{a})$ estimate. The *temporal-difference error* (TD Error) is the standard Q-learning training loss, and Farahmand & Szepesvári [8] proposed a model selection algorithm based on minimizing TD error. The *discounted sum of advantages* ($\sum_t \gamma^t A^\pi$) relates the difference in values $V^{\pi_b}(\mathbf{s}) - V^\pi(\mathbf{s})$ to the sum of advantages $\sum_t \gamma^t A^\pi(\mathbf{s}, \mathbf{a})$ over data from $\pi_b$, and was proposed by Kakade & Langford [13] and Murphy [24]. Finally, the *Monte Carlo corrected error* (MCC Error) is derived by arranging the discounted sum of advantages into a squared error, and was used as a training objective by Quillen et al. [27]. The exact expression of each of these metrics is in Appendix C.

Each of these baselines represents a different way to measure how well $Q(\mathbf{s}, \mathbf{a})$ fits the true return. However, it is possible to learn a good policy $\pi$ even when $Q(\mathbf{s}, \mathbf{a})$ fits the data poorly. In Q-learning, it is common to define an argmax policy $\pi = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. The argmax policy for $Q^*(\mathbf{s}, \mathbf{a})$ is $\pi^*$, and $Q^*$ has zero TD error. But, applying any monotonic function to $Q^*(\mathbf{s}, \mathbf{a})$ produces a $Q'(\mathbf{s}, \mathbf{a})$, whose TD error is non-zero, but whose argmax policy is still $\pi^*$. A good OPE metric should rate $Q^*$ and $Q'$ identically. This motivates our proposed classification-based OPE metrics: since $\pi$'s behavior only depends on the relative differences in Q-value, it makes sense to directly contrast Q-values against each other, rather than compare error between the Q-values and episode return. Doing so lets us compare Q-functions whose $Q(\mathbf{s}, \mathbf{a})$ estimates are inaccurate. Fig. 1a visualizes the differences between the baseline metrics and classification metrics.

## 6 Experiments

In this section, we investigate the correlation of OPC and SoftOPC with true average return, and how they may be used for model selection with off-policy data. We compare the correlation of these metrics with the correlation of the baselines, namely the TD Error, Sum of Advantages, and the MCC Error (see Sect. 5) in a number of environments and generalization failure scenarios. For each experiment, a validation dataset $\mathcal{D}$ is collected with a behavior policy $\pi_b$, and state-action pairs $(\mathbf{s}, \mathbf{a})$ are labeled as effective whenever they appear in a successful trajectory. In line with Sect. 3.3, several Q-functions $Q(\mathbf{s}, \mathbf{a})$ are trained for each task. For each $Q(\mathbf{s}, \mathbf{a})$, we evaluate each metric over $\mathcal{D}$ and true return of the equivalent argmax policy. We report both the coefficient of determination $R^2$ of line of best fit and the Spearman's rank correlation coefficient $\xi$ [31]. Our results are summarized in

Table 1. Our OPC/SoftOPC metrics are implemented using $p(y = 1) = 1$, as explained in Sect. 3 and Appendix D.

### 6.1 Simple Environments

**Binary tree.** As a didactic toy example, we used a binary tree MDP with depth of episode length $T$. In this environment,[2] each node is a state $\mathbf{s}_t$ with $r_t = 0$, unless it is a leaf/terminal state with reward $r_T \in \{0, 1\}$. Actions are {'left', 'right'}, and transitions are deterministic. We experimented with two extreme versions of this environment: *(a)* **1-Failure:** where the agent is successful unless it reaches the single failure leaf with $r_T = 0$; and *(b)* **1-Success:** where the agent fails unless it reaches the single success leaf with $r_T = 1$. In our experiments we used a full binary tree of depth $T = 6$. The initial state distribution was uniform over all non-leaf nodes, which means that the initial state could sometimes be initialized to one where failure is inevitable. The validation dataset $\mathcal{D}$ was collected by generating 1,000 episodes from a uniformly random policy. For the policies we wanted to evaluate, we generated 1,000 random Q-functions by sampling $Q(\mathbf{s}, \mathbf{a}) \sim U[0, 1]$ for every $(\mathbf{s}, \mathbf{a})$, defining the policy as $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. We compared the correlation of the actual on-policy performance of the policies with the scores given by the OPC, SoftOPC and the baseline metrics using $\mathcal{D}$, as shown in Table 1. SoftOPC correlates best and OPC correlates second best.

**Pong.** As we are specifically motivated by image-based tasks with binary rewards, the Atari [2] Pong game was a good choice for a simple environment that can have these characteristics. The visual input is of low complexity, and the game can be easily converted into a binary reward task by truncating the episode after the first point is scored. We learned Q-functions using DQN [23] and DDQN [36], varying hyperparameters such as the learning rate, the discount factor $\gamma$, and the batch size, as discussed in detail in Appendix E.2. A total of 175 model checkpoints are chosen from the various models for evaluation, and true average performance is evaluated over 3,000 episodes for each model checkpoint. For the validation dataset we used 38 Q-functions that were partially-trained with DDQN and generated 30 episodes from each, for a total of 1140 episodes. Similarly with the Binary Tree environments we compare the correlations of our metrics and the baselines to the true average performance over a number of on-policy episodes. As we show in Table 1, both our metrics outperform the baselines, OPC performs better than SoftOPC in terms of $R^2$ correlation but is similar in terms of Spearman correlation $\xi$.

Table 1: Summarized results of Experiments section. For each metric (leftmost column), we report $R^2$ of line of best fit and Spearman rank correlation coefficient $\xi$ for each environment (top row). These are: the binary tree and Pong tasks from Sect. 6.1, simulated grasping with train or test objects, and real-world grasping from Sect. 6.2. Baseline metrics are discussed in Sect. 5, and our metrics (OPC, SoftOPC) are discussed in Sect. 3. Occasionally, some baselines correlate well, but our proposed metrics (last two rows) are consistently among the best metrics for each environment.

| | **Tree (1 Fail)** | | **Tree (1 Succ)** | | **Pong** | | **Sim Train** | | **Sim Test** | | **Real-World** | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | $R^2$ | $\xi$ | $R^2$ | $\xi$ | $R^2$ | $\xi$ | $R^2$ | $\xi$ | $R^2$ | $\xi$ | $R^2$ | $\xi$ |
| **TD Err** | 0.01 | -0.13 | 0.02 | -0.15 | 0.05 | -0.18 | 0.02 | -0.37 | 0.10 | -0.51 | 0.17 | 0.48 |
| $\sum \boldsymbol{\gamma^t A^\pi}$ | 0.00 | 0.07 | 0.00 | 0.00 | 0.09 | -0.32 | **0.74** | 0.81 | **0.74** | **0.78** | 0.12 | 0.50 |
| **MCC Err** | 0.02 | -0.17 | 0.06 | -0.26 | 0.04 | -0.36 | 0.00 | 0.33 | 0.06 | -0.44 | 0.01 | -0.15 |
| **OPC** | 0.21 | 0.48 | **0.21** | 0.50 | **0.50** | 0.72 | 0.49 | **0.86** | 0.35 | 0.66 | 0.81 | 0.87 |
| **SoftOPC** | **0.23** | **0.53** | 0.19 | **0.51** | 0.36 | **0.75** | 0.55 | 0.76 | 0.48 | 0.77 | **0.91** | **0.94** |

### 6.2 Vision-based Robotic Grasping

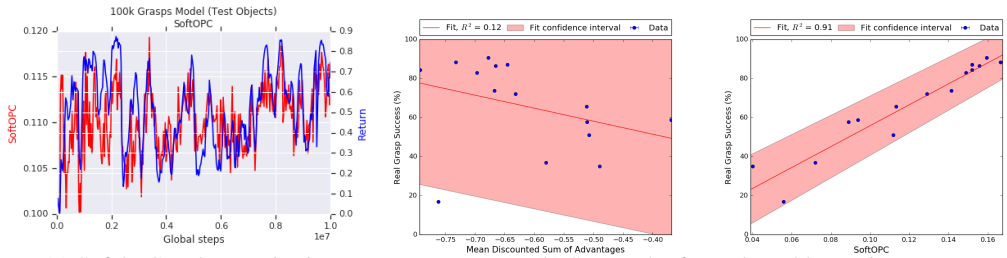
Our main experimental results were on simulated and real versions of a robotic environment and a vision-based grasping task, following the setup from Kalashnikov et al. [14], the details of which we briefly summarize. The observation at each time-step is a $472 \times 472$ RGB image from a camera placed over the shoulder of a robotic arm, of the robot and a bin of objects, as shown in Fig. 1b. At the start of an episode, objects are randomly dropped in a bin in front of the robot. The goal is to grasp any of the objects in that bin. Actions include continuous Cartesian displacements of the gripper, and the rotation of the gripper around the z-axis. The action space also includes three discrete commands: "open gripper", "close gripper", and "terminate episode". Rewards are sparse, with $r(\mathbf{s}_T, \mathbf{a}_T) = 1$

[2]Code for the binary tree environment is available at `https://bit.ly/2Qx6TJ7`.

if any object is grasped and 0 otherwise. All models are trained with the fully off-policy QT-Opt algorithm as described in Kalashnikov et al. [14].

In simulation we define a training and a test environment by generating two distinct sets of 5 objects that are used for each, shown in Fig. 3. In order to capture the different possible generalization failure scenarios discussed in Sect. 4, we trained Q-functions in a fully off-policy fashion with data collected by a hand-crafted policy with a 60% grasp success rate and $\epsilon$-greedy exploration (with $\epsilon$=0.1) with two different datasets both from the training environment. The first consists of $100,000$ episodes, with which we can show we have *insufficient off-policy training data* to perform well even in the training environment. The second consists of $900,000$ episodes, with which we can show we have sufficient data to perform well in the training environment, but due to *mismatched off-policy training data* we can show that the policies do not generalize to the test environment (see Fig. 3 for objects and Appendix E.3 for the analysis). We saved policies at different stages of training which resulted in 452 policies for the former case and 391 for the latter. We evaluated the true return of these policies on 700 episodes on each environment and calculated the correlation with the scores assigned by the OPE metrics based on held-out validation sets of $50,000$ episodes from the training environment and $10,000$ episodes from the test one, which we show in Table 1.

The real-world version of the environment has objects that were never seen during training (see Fig. 1b and 9). We evaluated 15 different models, trained to have varying degrees of robustness to the *training and test domain gap*, based on domain randomization and randomized–to-canonical adaptation networks [11].[3] Out of these, 7 were trained on-policy purely in simulation. True average return was evaluated over 714 episodes with 7 different sets of objects, and true policy real-world performance ranged from 17% to 91%. The validation dataset consisted of $4,000$ real-world episodes, 40% of which were successful grasps and the objects used for it were separate from the ones used for final evaluation used for the results in Table 1.




(a) SoftOPC and return in sim (b) Scatterplot for real-world grasping

Figure 4: **(a): SoftOPC in simulated grasping.** Overlay of SoftOPC (red) and return (blue) in simulation for model trained with 100k grasps. SoftOPC tracks episode return. **(b): Scatterplots for OPE metrics and real-world grasp success.** Scatterplots for $\sum \gamma^{t'} A^{\pi}(\mathbf{s}_{t'}, \mathbf{a}_{t'})$ (left) and SoftOPC (right) for the Real-World grasping task. Each point is a different grasping model. Shaded regions are a 95% confidence interval. $\sum \gamma^{t'} A^{\pi}(\mathbf{s}_{t'}, \mathbf{a}_{t'})$ works in simulation but fails on real data, whereas SoftOPC works well in both.

### 6.3 Discussion

Table 1 shows $R^2$ and $\xi$ for each metric for the different environments we considered. Our proposed SoftOPC and OPC consistently outperformed the baselines, with the exception of the simulated robotic test environment, on which the SoftOPC performed almost as well as the discounted sum of advantages on the Spearman correlation (but worse on $R^2$). However, we show that SoftOPC more reliably ranks policies than the baselines for real-world performance without any real-world interaction, as one can also see in Fig. 4b. The same figure shows the sum of advantages metric that works well in simulation performs poorly in the real-world setting we care about. Appendix F includes additional experiments showing correlation mostly unchanged on different validation datasets.

Furthermore, we demonstrate that SoftOPC can track the performance of a policy acting in the simulated grasping environment, as it is training in Fig. 4a, which could potentially be useful for early stopping. Finally, SoftOPC seems to be performing slightly better than OPC in most of the experiments. We believe this occurs because the Q-functions compared in each experiment tend to

[3]For full details of each of the models please see Appendix E.4.

have similar magnitudes. Preliminary results in Appendix H suggest that when Q-functions have different magnitudes, OPC might outperform SoftOPC.

## 7 Conclusion and future work

We proposed OPC and SoftOPC, classification-based off-policy evaluation metrics that can be used together with Q-learning algorithms. Our metrics can be used with binary reward tasks: tasks where each episode results in either a failure (zero return) or success (a return of one). While this class of tasks is a substantial restriction, many practical tasks actually fall into this category, including the real-world robotics tasks in our experiments. The analysis of these metrics shows that it can approximate the expected return in deterministic binary reward MDPs. Empirically, we find that OPC and the SoftOPC variant correlate well with performance across several environments, and predict generalization performance across several scenarios. including the simulation-to-reality scenario, a critical setting for robotics. Effective off-policy evaluation is critical for real-world reinforcement learning, where it provides an alternative to expensive real-world evaluations during algorithm development. Promising directions for future work include developing a variant of our method that is not restricted to binary reward tasks, and extending the analysis to stochastic tasks. However, even in the binary setting, we believe that methods such as ours can provide for a substantially more practical pipeline for evaluating transfer learning and off-policy reinforcement learning algorithms.

**Acknowledgements**

We would like to thank Razvan Pascanu, Dale Schuurmans, George Tucker, and Paul Wohlhart for valuable discussions.

## References

[1] Babaeizadeh, M., Finn, C., Erhan, D., Campbell, R. H., and Levine, S. Stochastic variational video prediction. In *International Conference on Representation Learning*, 2018.

[2] Bellemare, M. G., Naddaf, Y., Veness, J., and Bowling, M. The arcade learning environment: An evaluation platform for general agents. *Journal of Artificial Intelligence Research*, 47: 253–279, 2013.

[3] Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., and Zaremba, W. Openai gym. *arXiv preprint arXiv:1606.01540*, 2016.

[4] Cobbe, K., Klimov, O., Hesse, C., Kim, T., and Schulman, J. Quantifying generalization in reinforcement learning. *arXiv preprint arXiv:1812.02341*, 2018.

[5] Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. ImageNet: A Large-Scale Hierarchical Image Database. In *CVPR, 2009*, 2009.

[6] Dudik, M., Langford, J., and Li, L. Doubly robust policy evaluation and learning. In *ICML*, March 2011.

[7] Dudík, M., Erhan, D., Langford, J., Li, L., et al. Doubly robust policy evaluation and optimization. *Statistical Science*, 29(4):485–511, 2014.

[8] Farahmand, A.-M. and Szepesvári, C. Model selection in reinforcement learning. *Mach. Learn.*, 85(3):299–332, December 2011.

[9] Hanna, J. P., Stone, P., and Niekum, S. Bootstrapping with models: Confidence intervals for Off-Policy evaluation. In *Proceedings of the 16th Conference on Autonomous Agents and MultiAgent Systems*, AAMAS '17, pp. 538–546, Richland, SC, 2017. International Foundation for Autonomous Agents and Multiagent Systems.

[10] Horvitz, D. G. and Thompson, D. J. A generalization of sampling without replacement from a finite universe. *Journal of the American statistical Association*, 47(260):663–685, 1952.

[11] James, S., Wohlhart, P., Kalakrishnan, M., Kalashnikov, D., Irpan, A., Ibarz, J., Levine, S., Hadsell, R., and Bousmalis, K. Sim-to-real via sim-to-sim: Data-efficient robotic grasping via randomized-to-canonical adaptation networks. In *IEEE Conference on Computer Vision and Pattern Recognition*, March 2019.

[12] Jiang, N. and Li, L. Doubly robust off-policy value evaluation for reinforcement learning. November 2015.

[13] Kakade, S. and Langford, J. Approximately optimal approximate reinforcement learning. In *ICML*, 2002.

[14] Kalashnikov, D., Irpan, A., Pastor, P., Ibarz, J., Herzog, A., Jang, E., Quillen, D., Holly, E., Kalakrishnan, M., Vanhoucke, V., et al. Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:1806.10293*, 2018.

[15] Kiryo, R., Niu, G., du Plessis, M. C., and Sugiyama, M. Positive-unlabeled learning with non-negative risk estimator. In *NeurIPS*, pp. 1675–1685, 2017.

[16] Koos, S., Mouret, J.-B., and Doncieux, S. Crossing the reality gap in evolutionary robotics by promoting transferable controllers. In *Proceedings of the 12th annual conference on Genetic and evolutionary computation*, pp. 119–126. ACM, 2010.

[17] Koos, S., Mouret, J.-B., and Doncieux, S. The transferability approach: Crossing the reality gap in evolutionary robotics. *IEEE Transactions on Evolutionary Computation*, 17(1):122–145, 2012.

[18] Lee, A. X., Zhang, R., Ebert, F., Abbeel, P., Finn, C., and Levine, S. Stochastic adversarial video prediction. *arXiv preprint arXiv:1804.01523*, 2018.

[19] Lillicrap, T. P., Hunt, J. J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.

[20] Liu, Y., Gottesman, O., Raghu, A., Komorowski, M., Faisal, A., Doshi-Velez, F., and Brunskill, E. Representation balancing mdps for off-policy policy evaluation. In *NeurIPS*, 2018.

[21] Mahmood, A. R., van Hasselt, H. P., and Sutton, R. S. Weighted importance sampling for off-policy learning with linear function approximation. In *Advances in Neural Information Processing Systems*, pp. 3014–3022, 2014.

[22] Mannor, S., Simester, D., Sun, P., and Tsitsiklis, J. N. Bias and variance approximation in value function estimates. *Management Science*, 53(2):308–322, 2007.

[23] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529, 2015.

[24] Murphy, S. A. A generalization error for Q-Learning. *J. Mach. Learn. Res.*, 6:1073–1097, July 2005.

[25] Nichol, A., Pfau, V., Hesse, C., Klimov, O., and Schulman, J. Gotta learn fast: A new benchmark for generalization in rl. *arXiv preprint arXiv:1804.03720*, 2018.

[26] Precup, D., Sutton, R. S., and Singh, S. Eligibility traces for off-policy policy evaluation. In *Proceedings of the Seventeenth International Conference on Machine Learning, 2000*, pp. 759–766. Morgan Kaufmann, 2000.

[27] Quillen, D., Jang, E., Nachum, O., Finn, C., Ibarz, J., and Levine, S. Deep reinforcement learning for Vision-Based robotic grasping: A simulated comparative evaluation of Off-Policy methods. February 2018.

[28] Raghu, M., Irpan, A., Andreas, J., Kleinberg, R., Le, Q., and Kleinberg, J. Can deep reinforcement learning solve erdos-selfridge-spencer games? In *International Conference on Machine Learning*, pp. 4235–4243, 2018.

[29] Riedmiller, M., Hafner, R., Lampe, T., Neunert, M., Degrave, J., Van de Wiele, T., Mnih, V., Heess, N., and Springenberg, J. T. Learning by playing-solving sparse reward tasks from scratch. In *International Conference on Machine Learning*, 2018.

[30] Ross, S. and Bagnell, D. Efficient reductions for imitation learning. In *AISTATS*, pp. 661–668, 2010.

[31] S. Spearman, C. The proof and measurement of association between two things. *The American Journal of Psychology*, 15:72–101, 01 1904. doi: 10.2307/1412159.

[32] Theocharous, G., Thomas, P. S., and Ghavamzadeh, M. Personalized ad recommendation systems for life-time value optimization with guarantees. In *IJCAI*, pp. 1806–1812, 2015.

[33] Thomas, P. and Brunskill, E. Data-Efficient Off-Policy policy evaluation for reinforcement learning. In *International Conference on Machine Learning*, pp. 2139–2148, June 2016.

[34] Thomas, P. S., Theocharous, G., and Ghavamzadeh, M. High-Confidence Off-Policy evaluation. *AAAI*, 2015.

[35] Todorov, E., Erez, T., and Tassa, Y. Mujoco: A physics engine for model-based control. In *Intelligent Robots and Systems (IROS), 2012 IEEE/RSJ International Conference on*, pp. 5026–5033. IEEE, 2012.

[36] van Hasselt, H., Guez, A., and Silver, D. Deep reinforcement learning with double q-learning. In *Thirtieth AAAI Conference on Artificial Intelligence*, 2016.

[37] Zhang, A., Ballas, N., and Pineau, J. A dissection of overfitting and generalization in continuous reinforcement learning. June 2018.

[38] Zhang, C., Vinyals, O., Munos, R., and Bengio, S. A study on overfitting in deep reinforcement learning. April 2018.

# Appendix for Off-Policy Evaluation of Generalization for Deep Q-Learning in Binary Reward Tasks

## A Classification error bound

### A.1 Trajectory $\tau$ return $1 \Leftrightarrow$ all $\mathbf{a}_t$ effective

For the forward direction, because $\tau$ ends in a success state, from any $\mathbf{s}_{t+1}$ the optimal policy $\pi^*$ could reach a success state, so all $(\mathbf{s}_t, \mathbf{a}_t)$ must be effective.

For the reverse direction, if all $(\mathbf{s}_t, \mathbf{a}_t)$ are effective, then $(\mathbf{s}_T, \mathbf{a}_T)$ must be effective. Since $\mathbf{s}_{T+1}$ is a terminal state with no further actions, for $(\mathbf{s}_T, \mathbf{a}_T)$ to be effective, we must have $r(\mathbf{s}_T, \mathbf{a}_T) = 1$.

### A.2 Proof of Theorem 1

By definition, $\pi$ succeeds if and only if at every $\mathbf{s}_t$, it selects an effective $\mathbf{a}_t$. Since $\rho^+_{t,\pi}$ is defined as the state distribution conditioned on $\mathbf{a}_1, \cdots, \mathbf{a}_{t-1}$ being effective, the failure rate can be written as

$$1 - R(\pi) = \sum_{t=1}^{T} p(\pi \text{ makes first mistake at time } t) \tag{6}$$

$$= \sum_{t=1}^{T} \epsilon_t \prod_{i=1}^{t-1} (1 - \epsilon_i) \tag{7}$$

$$\leq \sum_{t=1}^{T} \epsilon_t \leq T\epsilon \tag{8}$$

This gives $R(\pi) \geq 1 - T\epsilon$ as desired. The bound is tight when $\epsilon_1 = \epsilon_2 = \cdots = \epsilon_{T-1} = 0, \epsilon_T = T\epsilon$.

### A.3 Alternate proof connecting to behavioral cloning

Since every policy that picks only effective actions achieves the optimal reward of 1, and $\epsilon$ is defined as the 0-1 loss over states conditioned on not selecting a catastrophic action, we can view $\epsilon$ as the 0-1 behavior cloning loss to an expert policy $\pi^*$. In this section, we present an alternate proof based on behavioral cloning bounds from Ross & Bagnell [30].

Theorem 2.1 of Ross & Bagnell [30] proves a $O(T^2\epsilon)$ cost bound for general MDPs. This differs from the $O(T\epsilon)$ cost derived above. The difference in bound comes because Ross & Bagnell [30] derive their proof in a general MDP, whose cost is upper bounded by 1 at every timestep. If $\pi$ deviates from the expert, it receives cost 1 several times, once for every future timestep. In binary reward MDPs, we only receive this cost once, at the final timestep. Transforming the proof to incorporate our binary reward MDP assumptions lets us recover the $O(T\epsilon)$ upper bound from Appendix A.2. We briefly explain the updated proof, using notation from [30] to make the connection more explicit.

Define $\epsilon_t$ as the expected 0-1 loss at time $t$ for $\pi$ under the state distribution of $\pi^*$. Since $\rho^+_{t,\pi}$ corresponds to states $\pi$ visits conditioned on never picking a catastrophic action, this is the same as our definition of $\epsilon_t$. The MDP is defined by cost instead of reward: cost $C(\mathbf{s})$ of state $\mathbf{s}$ is 0 for all timesteps except the final one, where $C(\mathbf{s}) \in \{0, 1\}$. Let $p_t$ be the probability $\pi$ hasn't made a mistake (w.r.t $\pi^*$) in the first $t$ steps, $d_t$ be the state distribution conditioned on no mistakes in the first $t-1$ steps, and $d'_t$ be the state distribution conditioned on $\pi$ making at least 1 mistake. In a general MDP with $0 \leq C(\mathbf{s}) \leq 1$, total cost $J(\pi)$ is bounded by $J(\pi) \leq \sum_{t=1}^{T} [p_{t-1}\mathbb{E}_{d_t}[C_\pi(\mathbf{s})] + (1 - p_{t-1})]$, where the 1st term is cost while following the expert and the 2nd term is an upper bound of cost 1 if outside of the expert distribution. In a binary reward MDP, since $C(\mathbf{s}) = 0$ for all $t$ except $t = T$, we can ignore every term in the summation except the final one, giving

$$J(\pi) = p_{T-1}\mathbb{E}_{d_T}[C_\pi(\mathbf{s}_T)] + (1 - p_{T-1}) \tag{9}$$

Note $\mathbb{E}_{d_T}[C_\pi(\mathbf{s}_T)] = \epsilon_T$, and as shown in the original proof, $p_t \geq 1 - \sum_{i=1}^{t} \epsilon_i$. Since $p_{T-1}$ is a probability, we have $p_{T-1}\mathbb{E}_{d_T}[C_\pi(\mathbf{s}_T)] \leq \mathbb{E}_{d_T}[C_\pi(\mathbf{s}_T)]$, which recovers the $O(T\epsilon)$ bound, and again this is tight when $\epsilon_1 = \cdots = \epsilon_{T-1} = 0, \epsilon_T = T\epsilon$.

$$J(\pi) \leq \mathbb{E}_{d_T}[C_\pi(\mathbf{s}_T)] + \sum_{t=1}^{T-1} \epsilon_t = \sum_{t=1}^{T} \epsilon_T = T\epsilon \tag{10}$$

# B Efficiently computing the OPC

$$\text{OPC}(Q) = p(y=1)\mathbb{E}_{(\mathbf{s},\mathbf{a}),y=1}\left[1_{Q(s,a)>b}\right] - \mathbb{E}_{(\mathbf{s},\mathbf{a})}\left[1_{Q(s,a)>b}\right] \tag{11}$$

Suppose we have $N$ transitions, of which $N^+$ of them have positive labels. Imagine placing each $Q(\mathbf{s},\mathbf{a})$ on the number line. Each $Q(\mathbf{s},\mathbf{a})$ is annotated with a score, $-1/N$ for unlabeled transitions and $p(y=1)/N^+ - 1/N$ for positive labeled transitions. Imagine sliding a line from $b = -\infty$ to $b = \infty$. At $b = -\infty$, the OPC score is $p(y=1) - 1$. The OPC score only updates when this line passes some $Q(\mathbf{s},\mathbf{a})$ in our dataset, and is updated based on the score annotated at $Q(\mathbf{s},\mathbf{a})$. After sorting by Q-value, we can find all possible OPC scores in $O(N)$ time by moving $b$ from $-\infty$ to $\infty$, noting the updated score after each $Q(\mathbf{s},\mathbf{a})$ we pass. Given $\mathcal{D}$, we sort the $N$ transitions in $O(N \log N)$, annotate them appropriately, then compute the maximum over all OPC scores.

# C Baseline metrics

We elaborate on the exact expressions used for the baseline metrics. In all baselines, $\mathbf{a}_t^\pi$ is the on-policy action $\arg\max_{\mathbf{a}} Q^\pi(\mathbf{s}_t,\mathbf{a})$.

**Temporal-difference error** The TD error is the squared error between $Q(\mathbf{s},\mathbf{a})$ and the 1-step return estimate of the action's value.

$$\mathbb{E}_{\mathbf{s}_t,\mathbf{a}_t\sim\pi_b}\left[\left(Q^\pi(\mathbf{s}_t,\mathbf{a}_t) - (r_t + \gamma Q^\pi(\mathbf{s}_{t+1},\mathbf{a}_t^\pi))\right)^2\right] \tag{12}$$

**Discounted sum of advantages** The difference of the value functions of two policies $\pi$ and $\pi_b$ at state $\mathbf{s}_t$ is given by the discounted sum of advantages [13, 24] of $\pi$ on episodes induced by $\pi_b$:

$$V^{\pi_b}(\mathbf{s}_t) - V^\pi(\mathbf{s}_t) = \mathbb{E}_{\mathbf{s}_t,\mathbf{a}_t\sim\pi_b}\left[\sum_{t'=t}^{T} \gamma^{t'-t} A^\pi(\mathbf{s}_{t'},\mathbf{a}_{t'})\right], \tag{13}$$

where $\gamma$ is the discount factor and $A^\pi$ the advantage function for policy $\pi$, defined as $A^\pi(\mathbf{s}_t,\mathbf{a_t}) = Q^\pi(\mathbf{s}_t,\mathbf{a}_t) - Q^\pi(\mathbf{s}_t,\mathbf{a}_t^\pi)$. Since $V^{\pi_b}$ is fixed, estimating (13) is sufficient to compare $\pi_1$ and $\pi_2$. The $\pi$ with smaller score is better.

$$\mathbb{E}_{\mathbf{s}_t,\mathbf{a}_t\sim\pi_b}\left[\sum_{t'=t}^{T} \gamma^{t'-t} A^\pi(\mathbf{s}_{t'},\mathbf{a}_{t'})\right]. \tag{14}$$

**Monte-Carlo estimate corrected with the discounted sum of advantages** Estimating $V^{\pi_b}(\mathbf{s}_t) = \mathbb{E}_{\pi_b}[\sum_{t'} \gamma^{t'-t} r_{t'}]$ with the Monte-Carlo return, substituting into Eqn. (13), and rearranging gives

$$V^\pi(\mathbf{s}_t) = \mathbb{E}_{\mathbf{s}_t,\mathbf{a}_t\sim\pi_b}\left[\sum_{t'=t}^{T} \gamma^{t'-t}\left(r_{t'} - A^\pi(\mathbf{s}_{t'},\mathbf{a}_{t'})\right)\right] \tag{15}$$

With $V^\pi(\mathbf{s}_t) + A^\pi(\mathbf{s}_t,\mathbf{a}_t) = Q^\pi(\mathbf{s}_t,\mathbf{a}_t)$, we can obtain an approximate $\widetilde{Q}$ estimate depending on the whole episode:

$$\widetilde{Q}_{MCC}(\mathbf{s}_t,\mathbf{a}_t,\pi) = \mathbb{E}_{\mathbf{s}_t,\mathbf{a}_t\sim\pi_b}\left[r_t + \sum_{t'=t+1}^{T} \gamma^{t'-t}(r_{t'} - A^\pi(\mathbf{s}_{t'},\mathbf{a}_{t'}))\right] \tag{16}$$

The MCC Error is the squared error to this estimate.

$$\mathbb{E}_{\mathbf{s}_t, \mathbf{a}_t \sim \pi_b} \left[ \left( Q^{\pi}(\mathbf{s}_t, \mathbf{a}_t) - \widetilde{Q}_{MCC}(\mathbf{s}_t, \mathbf{a}_t, \pi) \right)^2 \right] \tag{17}$$

Note that (17) was proposed before by Quillen et al. [27] as a training loss for a Q-learning variant, but not for the purpose of off-policy evaluation.

Eqn. (12) and Eqn. (16) share the same optimal Q-function, so assuming a perfect learning algorithm, there is no difference in information between these metrics. In practice, the Q-function will not be perfect due to errors in function approximation and optimization. Eqn. (16) is designed to rely on all future rewards from time $t$, rather than just $r_t$. We theorized that using more of the "ground truth" from $\mathcal{D}$ could improve the metric's performance in imperfect learning scenarios.

## D Argument for choosing $p(y = 1) = 1$

The positive class prior $p(y = 1)$ should intuitively depend on the environment, since some environments will have many more effective $(\mathbf{s}, \mathbf{a})$ than others. However, recall how error $\epsilon$ is defined. Each $\epsilon_t$ is defined as:

$$\epsilon_t = \mathbb{E}_{\rho^+_{t,\pi}} \left[ \sum_{\mathbf{a} \in \mathcal{A}_-(\mathbf{s}_t)} \pi(\mathbf{a}|\mathbf{s}_t) \right] \tag{18}$$

where state distribution $\rho^+_{t,\pi}$ is defined such that $\mathbf{a}_1, \cdots, \mathbf{a}_{t-1}$ were all effective. This is equivalent to following an optimal "expert" policy $\pi^*$, and although we are estimating $\epsilon_t$ from data generated by behavior policy $\pi_b$, we should match the positive class prior $p(y = 1)$ we would observe from expert $\pi^*$. Assuming the task is feasible, meaining the policy has effective actions available from the start, we have $R(\pi^*) = 1$. Therefore, although the validation dataset will likely have both successes and failures, a prior of $p(y = 1) = 1$ is the ideal prior, and this holds independently of the environment. As a didactic toy example, we show this holds for a binary tree domain. In this domain, each node is a state, actions are $\{left, right\}$, and leaf nodes are terminal with reward 0 or 1. We try $p(y = 1) \in \{0, 0.05, 0.1, \cdots, 0.9, 0.95, 1\}$ in two extremes: only 1 leaf fails, or only 1 leaf succeeds. Validation data is from the uniformly random policy. The frequency of effective $(\mathbf{s}, \mathbf{a})$ varies a lot between the two extremes, but in both Spearman correlation $\rho$ monotonically increases with $p(y = 1)$ and was best with $p(y = 1) = 1$. Fig. 5 shows Spearman correlation of OPC and SoftOPC with respect to $p(y = 1)$, when the tree is mostly success or failures. In both settings $p(y = 1) = 1$ has the best correlation.

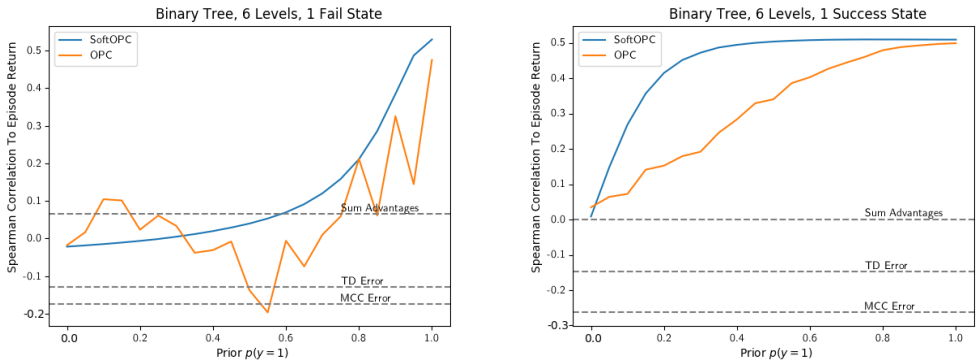



Figure 5: Spearman correlation of SoftOPC, OPC, and baselines with varying $p(y = 1)$. Baselines do not depend on $p(y = 1)$. Correlations further from 0 are better.

From an implementation perspective, $p(y = 1) = 1$ is also the only choice that can be applied across arbitrary validation datasets. Suppose $\pi_b$, the policy collecting our validation set, succeeds with probability $R(\pi_b) = p$. In the practical computation of OPC presented in Appendix B, we have $N$ transitions, and $pN$ of them have positive labels. Each $Q(\mathbf{s}, \mathbf{a})$ is annotated with a score: $\frac{-1}{N}$ for

unlabeled transitions and $\frac{p(y=1)}{pN} - \frac{1}{N}$ for positive labeled transitions. The maximal OPC score will be the sum of all annotations within the interval $[b, \infty)$, and we maximize over $b$.

For unlabeled transitions, the annotation is $-1/N$, which is negative. Suppose the annotation for positive transitions was negative as well. This occurs when $p(y = 1)/(pN) - 1/N < 0$. If every annotation is negative, then the optimal choice for $b$ is $b = \infty$, since the empty set has total 0 and every non-empty subset has a negative total. This gives $\text{OPC}(Q) = 0$, no matter what $Q(\mathbf{s}, \mathbf{a})$ we are evaluating, which makes the OPC score entirely independent of episode return.

This degenerate case is undesirable, and happens when $p(y = 1)/(pN) < 1/N$, or equivalently $p(y = 1) < p$. To avoid this, we should have $p(y = 1) \geq p = R(\pi_b)$. If we wish to pick a single $p(y = 1)$ that can be applied to data from arbitrary behavior policies $\pi_b$, then we should pick $p(y = 1) \geq R(\pi^*)$. In binary reward MDPs where $\pi^*$ can always succeed, this gives $p(y = 1) \geq R(\pi^*) = 1$, and since the prior is a probability, it should satisfy $0 \leq p(y = 1) \leq 1$, leaving $p(y = 1) = 1$ as the only option.

To complete the argument, we must handle the case where we have a binary reward MDP where $R(\pi^*) < 1$. In a binary reward MDP, the only way to have $R(\pi^*) < 1$ is if the sampled initial state $\mathbf{s}_1$ is one where $(\mathbf{s}_1, \mathbf{a})$ is catastrophic for all $\mathbf{a}$. From these $\mathbf{s}_1$, and all future $\mathbf{s}_t$ reachable from $\mathbf{s}_1$, the actions $\pi$ chooses do not matter - the final return will always be 0. Since $\epsilon_t$ is defined conditioned on only executing effective actions so far, it is reasonable to assume we only wish to compute the expectation over states where our actions can impact reward. If we computed optimal policy return $R(\pi^*)$ over just the initial states $\mathbf{s}_1$ where effective actions exist, we have $R(\pi^*) = 1$, giving $1 \leq p(y = 1) \leq 1$ once again.

# E Experiment details

## E.1 Binary tree environment details

The binary tree is a full binary tree with $k = 6$ levels. The initial state distribution is uniform over all non-leaf nodes. Initial state may sometimes be initialized to one where failure is inevitable. The validation dataset is collected by generating 1,000 episodes from the uniformly random policy. For Q-functions, we generate 1,000 random Q-functions by sampling $Q(\mathbf{s}, \mathbf{a}) \sim U[0, 1]$ for every $(\mathbf{s}, \mathbf{a})$, defining the policy as $\pi(\mathbf{s}) = \arg\max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. We try priors $p(y = 1) \in \{0, 0.05, 0.1, \cdots, 0.9, 0.95, 1\}$. Code for this environment is available at `https://gist.github.com/alexirpan/54ac855db7e0d017656645ef1475ac08`.

## E.2 Pong details

Fig. 6 is a scatterplot of our Pong results. Each color represents a different hyperparameter setting, as explained in the legend. From top to bottom, the abbreviations in the legend mean:

- `DQN`: trained with DQN
- `DDQN`: trained with Double DQN
- `DQN_gamma9`: trained with DQN, $\gamma = 0.9$ (default is 0.99).
- `DQN2`: trained with DQN, using a different random seed
- `DDQN2`: trained with Double DQN, using a different random seed
- `DQN_lr1e4`: trained with DQN, learning rate $10^{-4}$ (default is $2.5 \times 10^{-4}$).
- `DQN_b64`: trained with DQN, batch size 64 (default is 32).
- `DQN_fixranddata`: The replay buffer is filled entirely by a random policy, then a DQN model is trained against that buffer, without pushing any new experience into the buffer.
- `DDQN_fixranddata`: The replay buffer is filled entirely by a random policy, then a Double DQN model is trained against that buffer, without pushing new experience into the buffer.

In Fig. 6, models trained with $\gamma = 0.9$ are highlighted. We noticed that SoftOPC was worse at separating these models than OPC, suggesting the 0-1 loss is preferable in some cases. This is discussed further in Appendix H.

In our implementation, all models were trained in the full version of the Pong game, where the maximum return possible is 21 points. However, to test our approach we create a binary version for evaluation. Episodes in the validation set were truncated after the first point was scored. Return of the policy was estimated similarly: the policy was executed until the first point is scored, and the average return is computed over these episodes. Although the train time environment is slightly different from the evaluation environment, this procedure is fine for our method, since our method can handle environment shift and we treat $Q(\mathbf{s}, \mathbf{a})$ as a black-box scoring function. OPC can be applied as long as the validation dataset matches the semantics of the test environment where we estimate the final return.

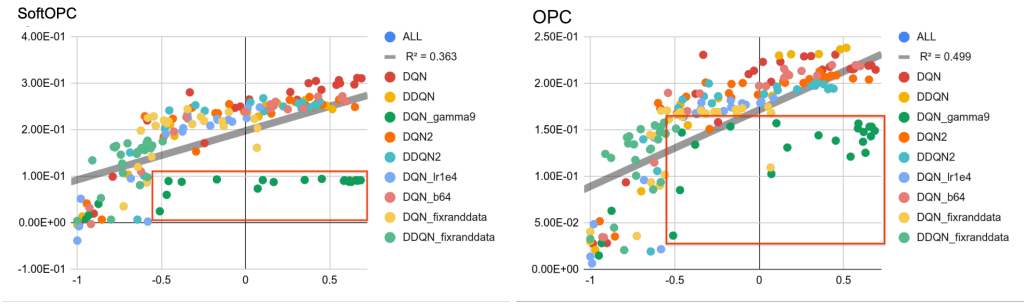
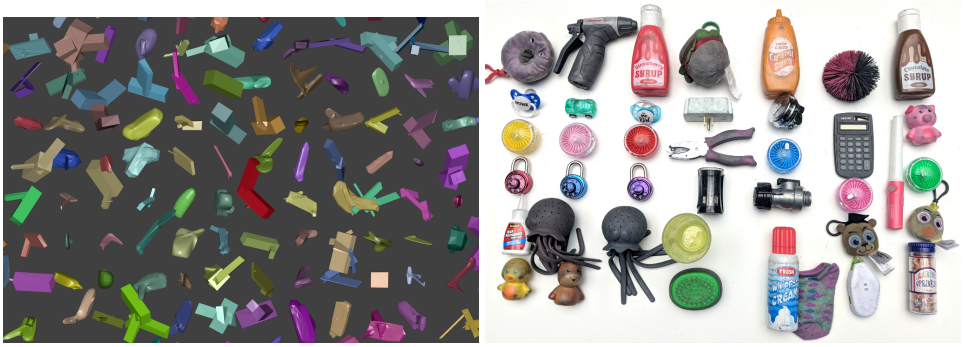



Figure 6: Scatterplot of episode return (x-axis) of Pong models against metric (y-axis), for SoftOPC (left) and OPC (right). Each color is a set of model checkpoints from a different hyperparameter setting, with the legend explaining the mapping from color to hyperparameters. In each plot, points trained with DQN, $\gamma = 0.9$ are boxed with a red rectangle. We observed that the hard 0-1 loss in OPC does a better job separating these models than the soft loss in SoftOPC.

### E.3 Simulated grasping details

The objects we use were generated randomly through procedural generation. The resulting objects are highly irregular and are often non-convex. Some example objects are shown in Fig. 7a.



(a) Procedurally generated objects for simulation (b) Test objects for real-world evaluation

Figure 7: **(a):** Example objects from the procedural generation process used during training in simulation. **(b):** Real test objects used during evaluation on real robots.

Fig. 8 demonstrates two generalization problems from Sect. 4: *insufficient off-policy training data* and *mismatched off-policy training data*. We trained two models with a limited 100k grasps dataset or a large 900k grasps dataset, then evaluated grasp success. The model with limited data fails to achieve stable grasp success due to overfitting to its limited dataset. Meanwhile, the model with abundant data learns to model the train objects, but fails to model the test objects, since they are unobserved at training time.

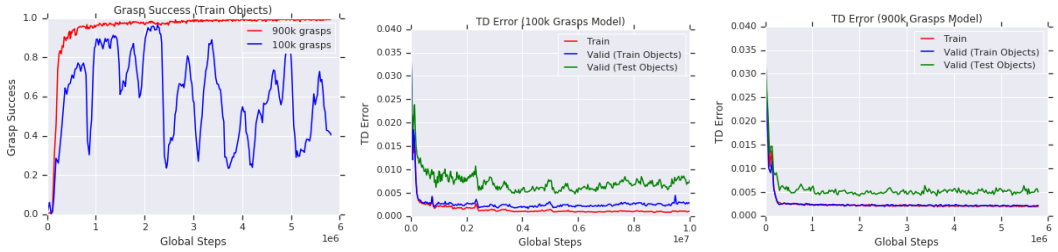



Figure 8: *Left:* Grasp success curve of model trained with 900k or 100k grasps. The 100k grasps model oscillates in performance. *Middle:* We see why: holdout TD error (blue) of the 100k grasps model is increasing. *Right*: The TD Error for the 900k grasp model is the same for train and holdout, but is still larger for test data on unseen test objects.

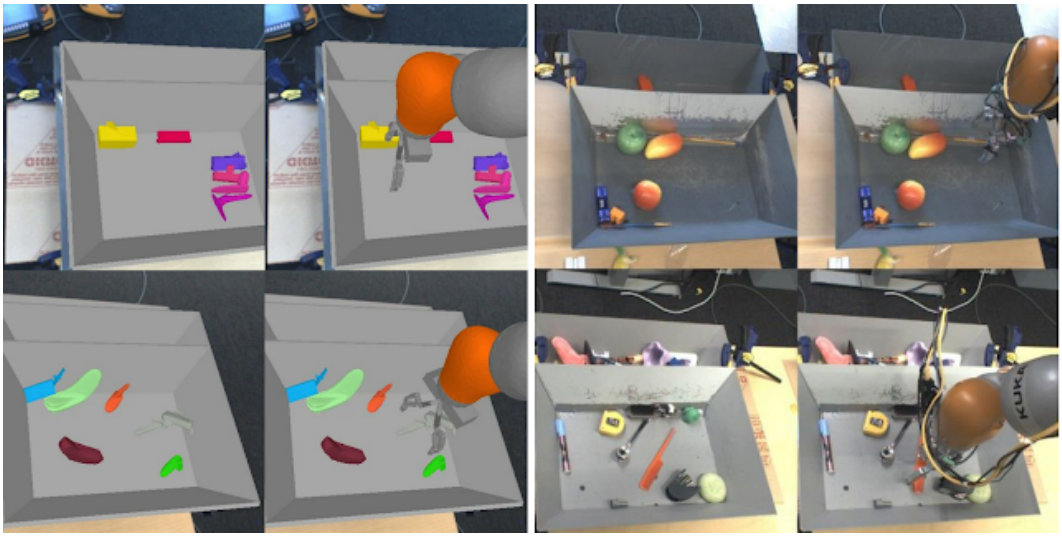


Figure 9: Visual differences for the robot grasping task between simulation (left) and reality (right). In simulation, models are trained to grasp procedurally generated shapes while in reality objects have more complex shapes. The simulated robot arm does not have the same colors and textures as the real robot and lacks the visible real cables. The tray in reality has a greater variation in appearance than in the simulation.

### E.4 Real-world grasping

Several visual differences between simulation and reality limit the real performance of model trained in simulation (see Fig. 9) and motivate simulation-to-reality methods such as the Randomized-to-Canonical Adaptation Networks (RCANs), as proposed by James et al. [11]. The 15 real-world grasping models evaluated were trained using variants of RCAN. These networks train a generator to transform randomized simulation images to a canonical simulated image. A policy is learned over this canonical simulated image. At test time, the generator transforms real images to the same canonical simulated image, facilitating zero-shot transfer. Optionally, the policy can be fine-tuned with real-world data, in this case the real-world training objects are distinct from the evaluation objects. The SoftOPC and real-world grasp success of each model is listed in Table 2. From top-to-bottom, the abbreviations mean:

- `Sim:` A model trained only in simulation.
- `Randomized Sim:` A model trained only in simulation with the *mild randomization* scheme from James et al. [11]: random tray texture, object texture and color, robot arm color, lighting direction and brightness, and one of 6 background images consisting of 6 different images from the view of the real-world camera.

- `Heavy Randomized Sim`: A model trained only in simulation with the *heavy randomization* scheme from James et al. [11]: every randomization from *Randomized Sim*, as well as slight randomization of the position of the robot arm and tray, randomized position of the divider within the tray (see Figure 1b in main text for a visual of the divider), and a more diverse set of background images.
- `Randomized Sim + Real (2k)`: The `Randomized Sim` Model, after training on an additional 2k grasps collected on-policy on the real robot.
- `Randomized Sim + Real (3k)`: The `Randomized Sim` Model, after training on an additional 3k grasps collected on-policy on the real robot.
- `Randomized Sim + Real (4k)`: The `Randomized Sim` Model, after training on an additional 4k grasps collected on-policy on the real robot.
- `Randomized Sim + Real (5k)`: The `Randomized Sim` Model, after training on an additional 5k grasps collected on-policy on the real robot.
- `RCAN`: The RCAN model, as described in [11], trained in simulation with a pixel level adaptation model.
- `RCAN + Real (2k)`: The `RCAN` model, after training on an additional 2k grasps collected on-policy on the real robot.
- `RCAN + Real (3k)`: The `RCAN` model, after training on an additional 3k grasps collected on-policy on the real robot.
- `RCAN + Real (4k)`: The `RCAN` model, after training on an additional 4k grasps collected on-policy on the real robot.
- `RCAN + Real (5k)`: The `RCAN` model, after training on an additional 5k grasps collected on-policy on the real robot.
- `RCAN + Dropout`: The `RCAN` model with dropout applied in the policy.
- `RCAN + InputDropout`: The `RCAN` model with dropout applied in the policy and RCAN generator.
- `RCAN + GradsToGenerator`: The `RCAN` model where the policy and RCAN generator are trained simultaneously, rather than training RCAN first and the policy second.

| **Model** | **SoftOPC** | **Real Grasp Success (%)** |
|---|---|---|
| Sim | 0.056 | 16.67 |
| Randomized Sim | 0.072 | 36.92 |
| Heavy Randomized Sim | 0.040 | 34.90 |
| Randomized Sim + Real (2k) | 0.129 | 72.14 |
| Randomized Sim + Real (3k) | 0.141 | 73.65 |
| Randomized Sim + Real (4k) | 0.149 | 82.92 |
| Randomized Sim + Real (5k) | 0.152 | 84.38 |
| RCAN | 0.113 | 65.69 |
| RCAN + Real (2k) | 0.156 | 86.46 |
| RCAN + Real (3k) | 0.166 | 88.34 |
| RCAN + Real (4k) | 0.152 | 87.08 |
| RCAN + Real (5k) | 0.159 | 90.71 |
| RCAN + Dropout | 0.112 | 51.04 |
| RCAN + InputDropout | 0.089 | 57.71 |
| RCAN + GradsToGenerator | 0.094 | 58.75 |

Table 2: Real-world grasping models used for Sect. 6.2 simulation-to-reality experiments.

# F SoftOPC performance on different validation datasets

For real grasp success we use 7 KUKA LBR IIWA robot arms to each make 102 grasp attempts from 7 different bins with test objects (see Fig. 7b). Each grasp attempt is allowed up to 20 steps and any grasped object is dropped back in the bin, a successful grasp is made if any of the test objects is held in the gripper at the end of the episode.

For estimating SoftOPC, we use a validation dataset collected from two policies, a poor policy with a success of 28%, and a better policy with a success of 51%. We divided the validation dataset based on the policy used, then evaluated SoftOPC on data from only the poor or good policy. Fig. 10 shows the correlation on these subsets of the validation dataset. The correlation is slightly worse on the poor dataset, but the relationship between SoftOPC and episode reward is still clear.

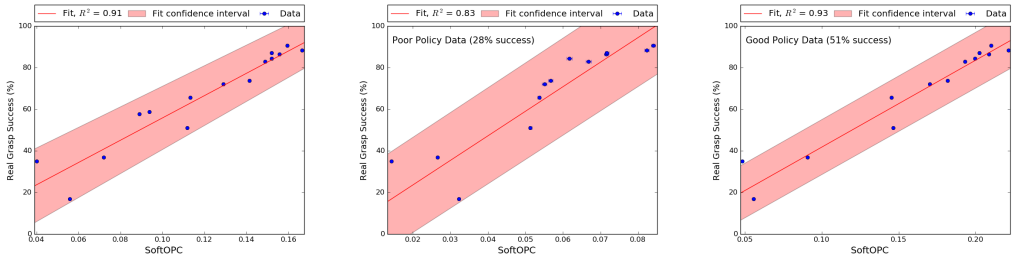



Figure 10: **SoftOPC versus the real grasp success over different validation datasets for Real-World grasping.** *Left*: SoftOPC over entire validation dataset. *Middle*: SoftOPC over validation data from only the poor policy (28% success rate). *Right*: SoftOPC over validation data from only the better policy (51% success). In each, a fitted regression line with its $R^2$ and 95% confidence interval is also shown.

As an extreme test of robustness, we go back to the simulated grasping environment. We collect a new validation dataset, using the same human-designed policy with $\epsilon = 0.9$ greedy exploration instead. The resulting dataset is almost all failures, with only 1% of grasps succeeding. However, this dataset also covers a broad range of states, due to being very random. Fig. 11 shows the OPC and SoftOPC still perform reasonably well, despite having very few positive labels. From a practical standpoint, this suggests that OPC or SoftOPC have some robustness to the choice of generation process for the validation dataset.

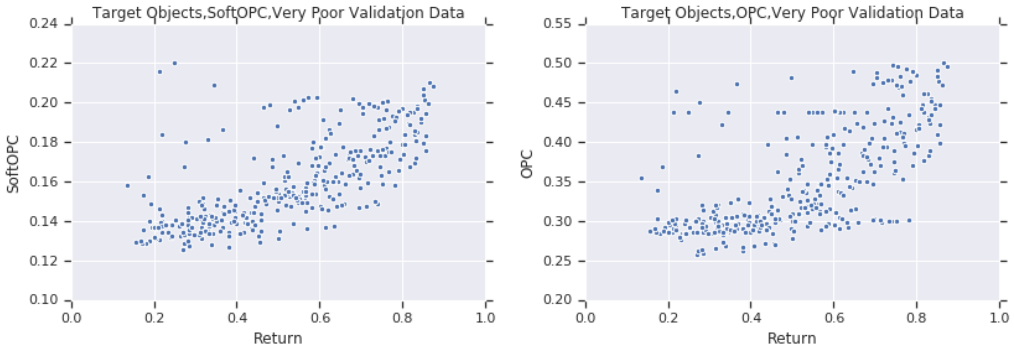



Figure 11: **SoftOPC and OPC over almost random validation data on test objects in simulated grasping.** We generate a validation dataset from an $\epsilon$-greedy policy where $\epsilon = 0.9$, leading to a validation dataset where only 1% of episodes succeed. *Left*: SoftOPC over the poor validation dataset. $R^2 = 0.83, \rho = 0.94$. *Right*: OPC over the poor validation dataset. $R^2 = 0.83, \rho = 0.88$.

## G Plots of Q-value distributions

In Fig. 12, we plot the Q-values of two real-world grasping models. The first is trained only in simulation and has poor real-world grasp success. The second is trained with a mix of simulated and real-world data. We plot a histogram of the average Q-value over each episode of validation set $\mathcal{D}$. The better model has a wider separation between successful Q-values and failed Q-values.

## H Comparison of OPC and SoftOPC

We elaborate on the argument presented in the main paper, that OPC performs better when $Q(\mathbf{s}, \mathbf{a})$ have different magnitudes, and otherwise SoftOPC does better. To do so, it is important to consider

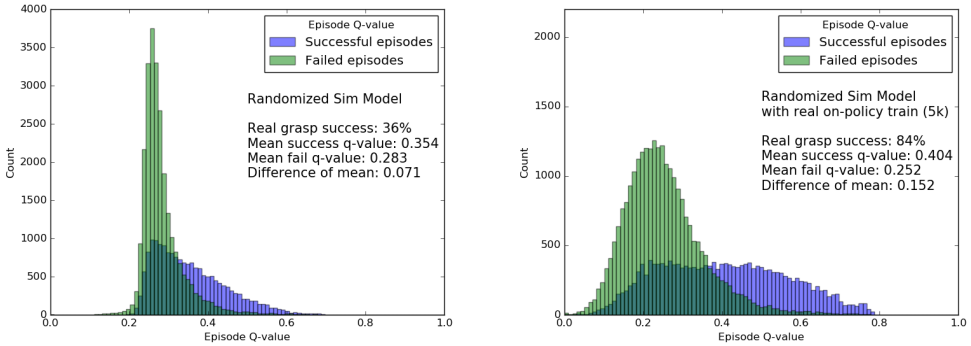
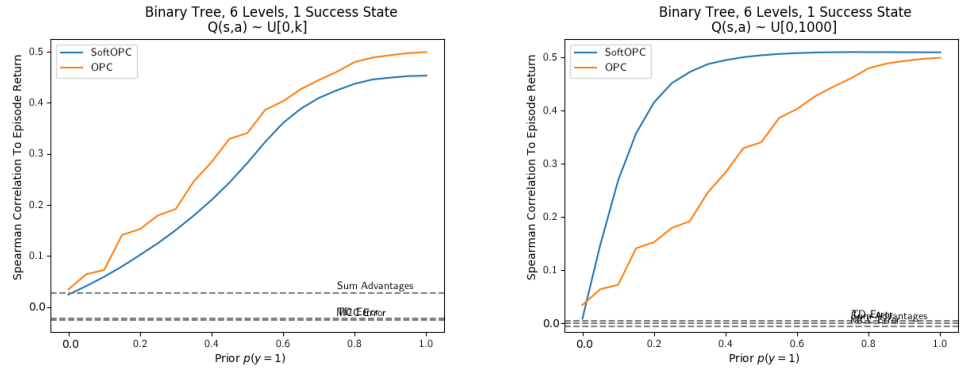



Figure 12: **Q-value distributions for successful and failed episodes.** *Left:* Q-value distributions over successful and failed episodes in an off-policy data-set according to a learned policy with a poor grasp success rate of 36%. *Right:* The same distributions after the learned policy is improved by 5,000 grasps of real robot data, achieving a 84% grasp success rate.




(a) $Q(\mathbf{s}, \mathbf{a}) \sim U[0, k]$ (b) $Q(\mathbf{s}, \mathbf{a}) \sim U[0, 1000]$

Figure 13: Spearman correlation in binary tree with one success state for different Q-function generation methods. Varying magnitudes between Q-functions causes the SoftOPC to perform worse.

how the Q-functions were trained. In the tree environments, $Q(\mathbf{s}, \mathbf{a})$ was sampled uniformly from $U[0, 1]$, so $Q(\mathbf{s}, \mathbf{a}) \in [0, 1]$ by construction. In the grasping environments, the network architecture ends in $\text{sigmoid}(x)$, so $Q(\mathbf{s}, \mathbf{a}) \in [0, 1]$. In these experiments, SoftOPC did better. In Pong, $Q(\mathbf{s}, \mathbf{a})$ was not constrained in any way, and these were the only experiments where discount factor $\gamma$ was varied between models. Here, OPC did better.

The hypothesis that Q-functions of varying magnitudes favor OPC can be validated in the tree environment. Again, we evaluate 1,000 Q-functions, but instead of sampling $Q(\mathbf{s}, \mathbf{a}) \sim U[0, 1]$, the $k$th Q-function is sampled from $Q(\mathbf{s}, \mathbf{a}) \sim U[0, k]$. This produces 1,000 different magnitudes between the compared Q-functions. Fig. 13a demonstrates that when magnitudes are deliberately changed for each Q-function, the SoftOPC performs worse, whereas the non-parametric OPC is unchanged. To demonstrate this is caused by a difference in magnitude, rather than large absolute magnitude, OPC and SoftOPC are also evaluated over $Q(\mathbf{s}, \mathbf{a}) \sim U[0, 1000]$. Every Q-function has high magnitude, but their magnitudes are consistently high. As seen in Fig. 13b, in this setting the SoftOPC goes back to outperforming OPC.

# I Scatterplots of Each Metric

We present scatterplots of each of the metrics in the simulated grasping environment from Sect. 6.2. We trained two Q-functions in a fully off-policy fashion, one with a dataset of $100,000$ episodes, and the other with a dataset of $900,000$ episodes. For every metric, we generate a scatterplot of all the

model checkpoints. Each model checkpoint is color coded by whether it was trained with $100,000$ episodes or $900,000$ episodes.

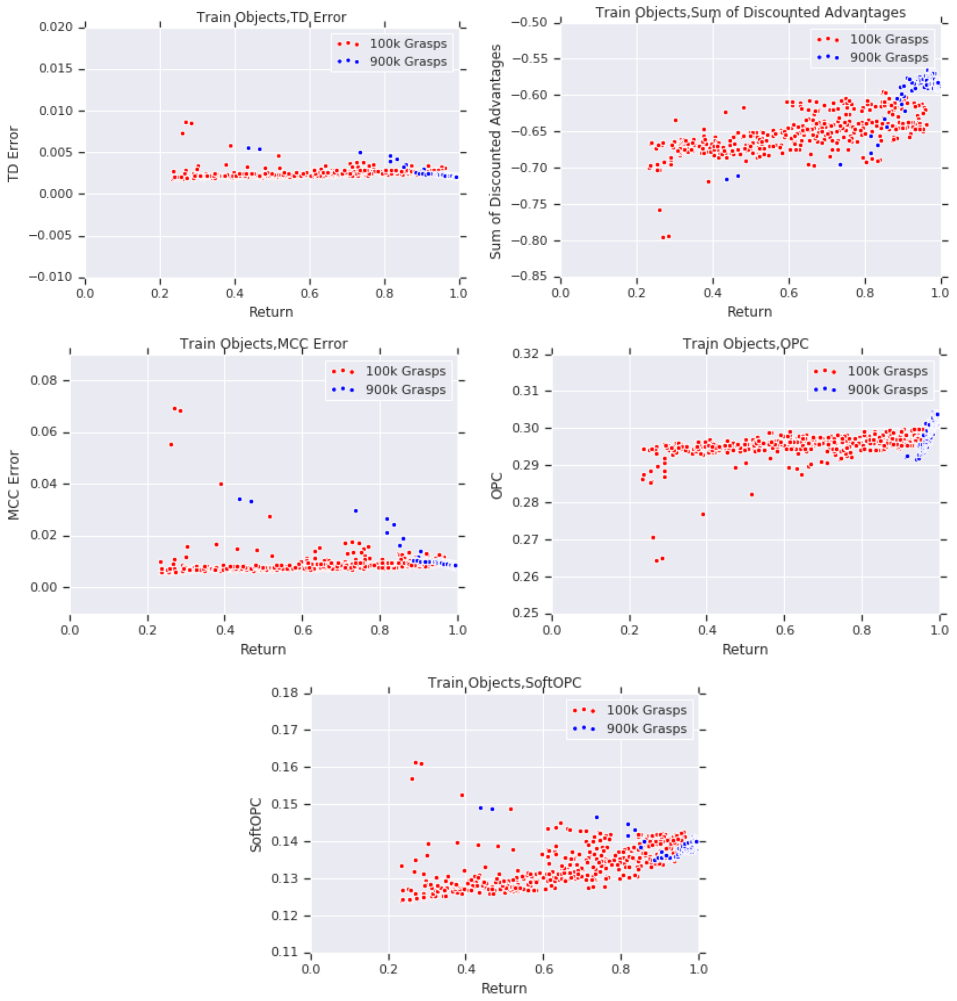



Figure 14: **Scatterplots of each metric in simulated grasping over train objects.** From left-to-right, top-to-bottom, we present scatterplots for: the TD error, $\sum \gamma^{t'} A^{\pi}(\mathbf{s}_{t'}, \mathbf{a}_{t'})$, MCC error, OPC, and SoftOPC.